

PROJECT DESIGN PHASE

ComicCraft – AI Comic Story Creator using Gemini Models

1. Design Objective

The design of ComicCraft is focused on creating a simple, interactive web application that converts user-provided story ideas into personalized comic stories. The design connects the frontend, FastAPI backend, AI models, image generation, comic layout, and PDF export into one continuous workflow.

2. Overall System Architecture

ComicCraft is designed around three primary components:

- Frontend – HTML, CSS, and Jinja2
- Backend – FastAPI application
- AI Integration – Google Gemini APIs and Hugging Face Diffusers

The frontend collects user information and displays the generated comic. The backend manages routing, data processing, AI workflow coordination, layout construction, and PDF export. The AI services generate the story content and comic illustrations.

3. Frontend Design

- index.html is designed as the main input page.
- The input form collects Story Prompt, Character Name, Setting, Story Tone, and Art Style.
- comic_preview.html displays the generated comic panel-by-panel.
- export_success.html provides confirmation after the comic is exported.
- The interface uses simple typography, organized form sections, centered layouts, padding, rounded corners, box shadows, and button hover effects.
- The design uses a scenic background and a light color scheme to create an inviting creative environment.
- Navigation is kept minimal so the user can move directly from story input to comic generation.

4. Backend Design

- FastAPI manages server-side operations and route handling.
- Form data is received from the frontend using FastAPI Form(...) parameters.
- JSON requests are validated using the Pydantic PromptRequest schema.
- The backend coordinates outline generation, story generation, image generation, layout building, and PDF export.
- HTTPException and try-except handling are used for unexpected failures such as API or JSON errors.

5. AI Integration Design

- Gemini Flash generates a structured 5-panel comic outline.
- Gemini Pro expands the outline into detailed narration and character dialogue.
- Stable Diffusion generates comic-style illustrations from image prompts.
- AI outputs are processed and passed to the Jinja2 templates for preview and export.
- Each comic is dynamically generated and customized from the user's input.

6. Core Functional Modules

generate_outline() – gemini_flash.py

Creates the structured 5-panel outline. Each panel contains a panel number, title, scene description, and image generation prompt.

generate_story() – gemini_pro.py

Expands the panel outlines into a complete comic-style story with narration and character dialogue.

generate_image() – image_generator.py

Generates comic-style images using Stable Diffusion and saves them in the static/panels directory.

build_comic_layout() – layout_builder.py

Matches generated images with their corresponding panel stories and creates the final layout structure.

save_pdf() – exporters.py

Compiles panel images and narration into a multi-page PDF using FPDF.

7. Data Flow Design

1. User enters the story prompt and preferences.
2. Frontend sends the information to the FastAPI backend.
3. Gemini Flash creates the structured 5-panel outline.
4. Gemini Pro generates narration and dialogue.
5. Stable Diffusion generates an illustration for each panel.
6. The backend combines the text and images into a comic layout.
7. The comic preview is rendered through Jinja2.
8. The user can export the completed comic as a PDF.
9. The export-success page confirms the completed download.

8. Route Design

- / – loads index.html and provides the comic input form.
- /generate – processes form submission and generates the comic preview.
- /generate-comic/json – accepts JSON input and returns comic layout data and the generated PDF path.

- /export-success – displays the export confirmation page.
- /test-image – tests image generation directly from a custom prompt.

9. Template Design

- index.html – input collection and submission to /generate.
- comic_preview.html – dynamically displays panel number, title, image, scene description, captions, and narration.
- export_success.html – displays confirmation and provides an option to return to the homepage.
- Jinja2 loops through the layout list to render each comic panel dynamically.

10. Comic Panel Design

- Each panel has a panel number and title.
- Each panel contains an AI-generated illustration.
- Each panel includes a scene description.
- Captions provide short environmental or background information.
- Narration presents the character's actions, emotions, or dialogue.
- The image prompt reference can also be displayed for the generated panel.

11. PDF Export Design

- The layout_builder.py module organizes the panel content before export.
- The exporters.py module uses FPDF to create the PDF.
- Panel images, titles, descriptions, captions, and narration are placed into the exported document.
- The PDF is saved in the static/exports folder.
- After export, the user is directed to the success page.

12. Development Environment Design

- Python and pip are required.
- A virtual environment is used to isolate project dependencies.
- Required libraries include FastAPI, Uvicorn, Jinja2, python-multipart, Google Generative AI, Diffusers, Transformers, FPDF, Pillow, and Accelerate.
- The local development server is started using Uvicorn.
- FastAPI documentation is available through the /docs endpoint.

13. Security and Configuration Design

- Sensitive Gemini API and Hugging Face credentials are stored using environment variables.
- A .env file can be used with python-dotenv.
- The project separates configuration and application code from secret API keys.

14. Deployment Design

- The application is initially designed for local deployment.

- A virtual environment manages project dependencies.
- The application is tested through the local FastAPI server.
- The design can later support deployment to a cloud platform.

15. Future Design Extensions

- User profiles and account management.
- Personal comic libraries.
- Multi-page story arcs.
- Additional character and art-style customization.
- Cloud-based deployment.
- More editing controls before PDF export.

16. Design Summary

The ComicCraft project design provides an end-to-end architecture connecting user input, AI story generation, image generation, comic layout, web presentation, and PDF export. The modular design keeps the frontend, backend, AI integration, and export functions clearly separated while allowing them to work together as a single comic-generation workflow.

Source: ComicCraft – AI Comic Story Creator using Gemini Models